

LAPORAN PRAKTIKUM

Modul 5

Disusun Untuk Memenuhi Tugas Kuliah

Mata Kuliah: Teknologi IoT

Dosen Pembimbing:

Agus Ambarwari, S.Pd., M.Kom.

Dr. Ir. Septafiansyah Dwi Putra



Oleh:

Alby Halim (25782031)

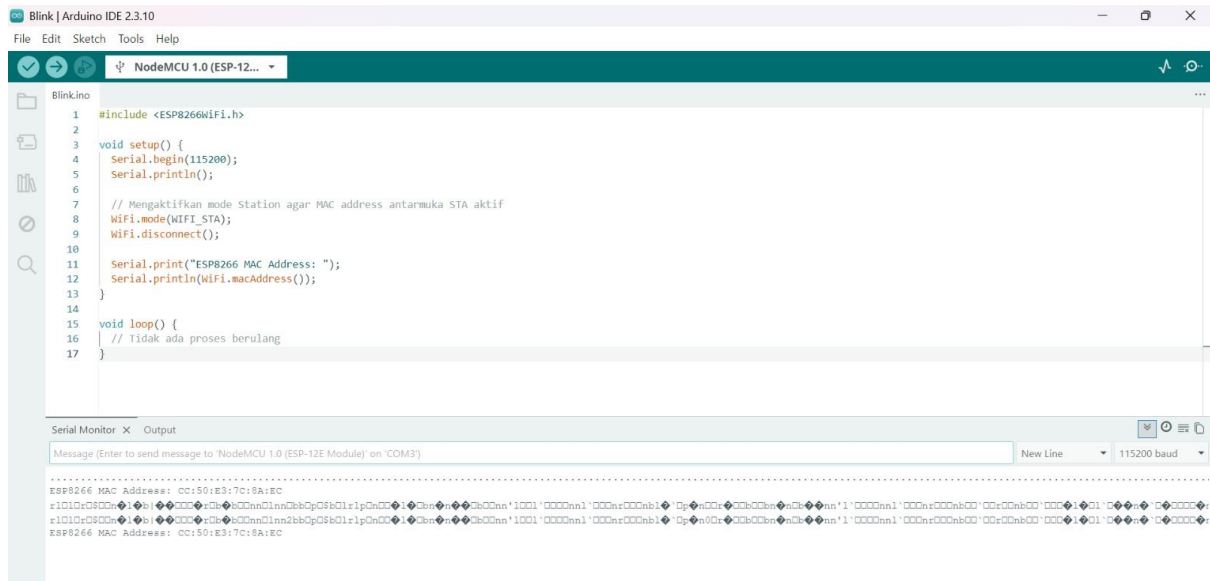
PROGRAM STUDI D4 TEKNOLOGI REKAYASA INTERNET

JURUSAN TEKNOLOGI INFORMASI

POLITEKNIK NEGERI LAMPUNG

PRAKTIKUM

5.5.1 Praktikum 1: Identifikasi MAC Address Perangkat NodeMCU



```
1 #include <ESP8266WiFi.h>
2
3 void setup() {
4   Serial.begin(115200);
5   Serial.println();
6
7   // Mengaktifkan mode Station agar MAC address antarmuka STA aktif
8   WiFi.mode(WIFI_STA);
9   WiFi.disconnect();
10
11   Serial.print("ESP8266 MAC Address: ");
12   Serial.println(WiFi.macAddress());
13 }
14
15 void loop() {
16   // Tidak ada proses berulang
17 }
```

Serial Monitor X Output

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200 baud

ESP8266 MAC Address: CC:50:E3:7C:8A:BC



Penjelasan Singkat Kode:

WiFi.mode(WIFI_STA); : Menyiapkan modul radio Wi-Fi internal ke dalam mode Station agar alamat perangkat keras (Hardware MAC) siap dialokasikan oleh SDK.

WiFi.disconnect(); : Memastikan mikrokontroler tidak mencoba melakukan proses penyambungan ke jaringan luar.

WiFi.macAddress(); : Mengambil string alamat fisik 48-bit bawaan pabrik yang bersifat unik secara global.

Praktikum 2: Komunikasi Nirkabel One-to-Many (ESP-NOW)

Input kode pengirim

```
Blink | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

Blinkino
1 #include <ESP8266WiFi.h>
2 #include <espnw.h>
3
4 // Struktur data penerima (wajib identik dengan sisi pengirim)
5 typedef struct struct_pesan {
6     int perintahId;
7     int nilaiParameter;
8 } struct_pesan;
9
10 struct_pesan paketTerima;
11
12 // Callback otomatis saat antenna mendeteksi paket masuk
13 void OnDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
14     char macStr[18];
15     snprintf(macStr, sizeof(macStr), "%02x:%02x:%02x:%02x:%02x:%02x",
16             mac_addr[0], mac_addr[1], mac_addr[2], mac_addr[3], mac_addr[4], mac_addr[5]);
17
18     // Salin buffer memori radio langsung ke struktur data variabel
19     memcpy(&paketTerima, incomingData, sizeof(paketTerima));
20
21     Serial.print("Paket masuk dari: ");
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 43

Paket masuk dari: 68:c6:3a:cd:12:d7
Ukuran Data: 8 byte
Perintah ID: 101
Nilai Parameter: 69

Input Kode penerima yang memonitoring

```
sketch_sep21a | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

sketch_sep21a.ino
1 #include <ESP8266WiFi.h>
2 #include <espnw.h>
3
4 // GANTI dengan MAC Address Board Penerima 1 dan Board Penerima 2 Anda!
5 uint8_t receiver1[] = {0xCC, 0x50, 0xE3, 0x7C, 0x8A, 0xEC};
6 uint8_t receiver2[] = {0x84, 0xF3, 0xEB, 0x85, 0xFE, 0xA0};
7 uint8_t receiver3[] = {0xB0, 0xDD, 0xC2, 0x75, 0x28, 0x15};
8
9 // Definisi struktur data paket instruksi
10 typedef struct struct_pesan {
11     int perintahId;
12     int nilaiParameter;
13 } struct_pesan;
14
15 struct_pesan paketKirim;
16
17 unsigned long previousMillis = 0;
18 const long interval = 2000; // Transmisi berkala setiap 2 detik
19
20 void setup() {
21     Serial.begin(115200);
22     WiFi.mode(WIFI_STA);
23     WiFi.begin("ESP-NOW", "ESP-NOW");
24     while (WiFi.status() != WL_CONNECTED) {
25         delay(500);
26         Serial.print(".");
27     }
28     Serial.println("WiFi connected");
29 }
30
31 void loop() {
32     if (millis() - previousMillis > interval) {
33         previousMillis = millis();
34         // Kirim paket ke receiver1
35         WiFiEspNow.send(receiver1, sizeof(receiver1), paketKirim);
36         Serial.print("Kirim paket ke: ");
37         for (int i = 0; i < sizeof(receiver1); i++) {
38             Serial.print(receiver1[i], HEX);
39             Serial.print(" ");
40         }
41         Serial.print(" | Status: Berhasil Diterima\n");
42         // Kirim paket ke receiver2
43         WiFiEspNow.send(receiver2, sizeof(receiver2), paketKirim);
44         Serial.print("Kirim paket ke: ");
45         for (int i = 0; i < sizeof(receiver2); i++) {
46             Serial.print(receiver2[i], HEX);
47             Serial.print(" ");
48         }
49         Serial.print(" | Status: Berhasil Diterima\n");
50         // Kirim paket ke receiver3
51         WiFiEspNow.send(receiver3, sizeof(receiver3), paketKirim);
52         Serial.print("Kirim paket ke: ");
53         for (int i = 0; i < sizeof(receiver3); i++) {
54             Serial.print(receiver3[i], HEX);
55             Serial.print(" ");
56         }
57         Serial.print(" | Status: Berhasil Diterima\n");
58     }
59 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

10:42:21.530 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:42:21.530 -> Kirim paket ke: bc:dd:c2:75:28:15 | Status: Berhasil Diterima
10:42:23.524 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:42:23.524 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:42:23.524 -> Kirim paket ke: bc:dd:c2:75:28:15 | Status: Berhasil Diterima
10:42:25.561 -> Kirim paket ke: cc:50:e3:7c:8a:ec | Status: Berhasil Diterima
10:42:25.561 -> Kirim paket ke: 84:f3:eb:85:fe:a0 | Status: Berhasil Diterima
10:42:25.561 -> Kirim paket ke: bc:dd:c2:75:28:15 | Status: Berhasil Diterima

Ln 54, Col 29 NodeMCU 1.0 (ESP-12E Module) on COM3 6

Penjelasan Fungsi Khusus ESP-NOW:

`Esp_now_init()` : Menginisialisasi firmware stack ESP-NOW pada chip ESP8266.

Mengembalikan nilai 0 jika berhasil.

`Esp_now_set_self_role()` : Mengonfigurasi peran operasional NodeMCU

(`ESP_NOW_ROLE_CONTROLLER` untuk pengirim, `ESP_NOW_ROLE_SLAVE` untuk penerima).

`Esp_now_register_send_cb(OnDataSent)` : Mendaftarkan fungsi callback yang akan dipanggil sesaat setelah gelombang radio memancarkan paket untuk memastikan laporan ACK diterima atau gagal.

`Esp_now_register_recv_cb(OnDataRecv)` : Mendaftarkan fungsi callback interupsi saat paket radio masuk ke antena penerima.

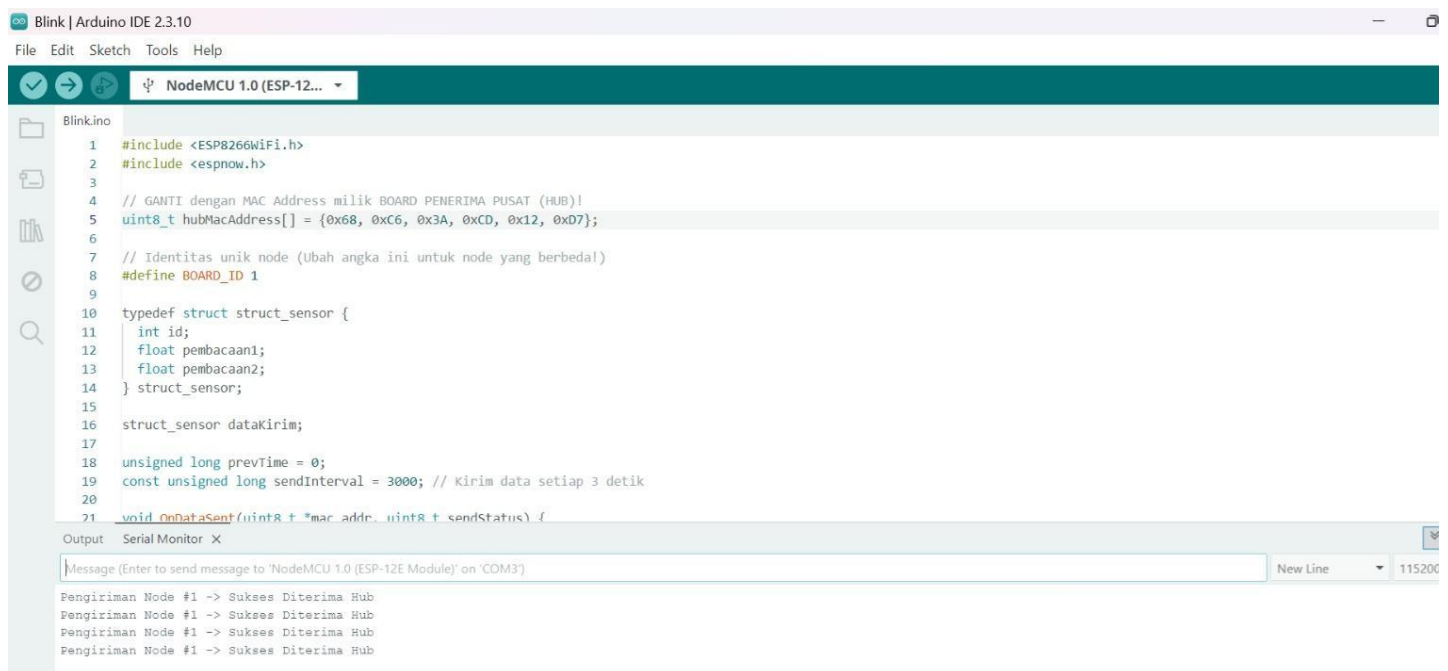
`Esp_now_add_peer(mac, role, channel, key, key_len)` : Mendaftarkan perangkat tujuan ke dalam tabel routing memori chip.

`Esp_now_send(0, data, size)` : Mengirimkan muatan data. Memasukkan angka 0 pada argumen pertama berarti paket dikirimkan berurutan ke seluruh peer yang terdaftar.

`Memcpy(&paketTerima, incomingData, sizeof(paketTerima))` : Menyalin deretan byte mentah dari penyangga memori (buffer) ke dalam variabel struktur `paketTerima`.

5.5.3 Praktikum 3: Pengumpulan Data Telemetry Many-to-One (ESP-NOW)

Pengirim



```
Blink.ino
1  #include <ESP8266WiFi.h>
2  #include <espnow.h>
3
4  // GANTI dengan MAC Address milik BOARD PENERIMA PUSAT (HUB)!
5  uint8_t hubMacAddress[] = {0x68, 0xC6, 0x3A, 0xCD, 0x12, 0xD7};
6
7  // Identitas unik node (Ubah angka ini untuk node yang berbeda!)
8  #define BOARD_ID 1
9
10 typedef struct struct_sensor {
11     int id;
12     float pembacaan1;
13     float pembacaan2;
14 } struct_sensor;
15
16 struct_sensor dataKirim;
17
18 unsigned long prevTime = 0;
19 const unsigned long sendInterval = 3000; // Kirim data setiap 3 detik
20
21 void OnDataSent(uint8_t *mac_addr, uint8_t sendStatus) {
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

New Line 115200

Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub
Pengiriman Node #1 -> Sukses Diterima Hub

Penerima

```
sketch_sep21a.ino
1 #include <ESP8266WiFi.h>
2 #include <espnow.h>
3
4 typedef struct struct_sensor {
5     int id;
6     float pembacaan1;
7     float pembacaan2;
8 } struct_sensor;
9
10 struct_sensor dataTerima;
11
12 // Array penampung data terakhir: Index 0 untuk Node 1, Index 1 untuk Node 2
13 struct_sensor records[2];
14
15 void onDataRecv(uint8_t *mac_addr, uint8_t *incomingData, uint8_t len) {
16     memcpy(&dataTerima, incomingData, sizeof(dataTerima));
17
18     // Validasi ID agar tidak terjadi error index array out-of-bounds
19     if (dataTerima.id < 0 || dataTerima.id > 1) {
20         return;
21     }
22 }
```

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

10:51:15.565 -> Parameter 1 : 27.10
10:51:15.565 -> Parameter 2 : 61.40
10:51:16.628 -> DATA TERBARU DARI NODE #1
10:51:16.628 -> Parameter 1 : 26.50
10:51:16.628 -> Parameter 2 : 48.40

Ln 15, Col 58 NodeMCU 1.0 (ESP-12E Module) on COM3

5.5.4 Praktikum 4: Jaringan Sensor Ad-Hoc Multi-Hop (painlessMesh)

```
Blink.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "LabSRAR" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "123" // Kata sandi jaringan Mesh
7 #define MESH_PORT 8080 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Ganti DHT22 jika menggunakan varian DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-2";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototype fungsi pengiriman pesan berkala
21 void sendMessage();
```

Library Manager

painless

Type: All
Topic: All

Alteriom PainlessMesh by Coopdis,Scotty...
A painless way to setup a mesh with ESP8266 and ESP32 devices with Alteriom extensions painlessMesh...
2.1.1 INSTALL

Painless Mesh by Coopdis,Scotty...
1.5.7 installed
A painless way to setup a mesh with ESP8266 and ESP32 devices with painlessMesh is a user-friendly...
1.5.7 REMOVE

SparkFun LSM9DS1 IMU by SparkFun Electronics...
A driver library for the LSM9DS1 IMU. Communicates with the LSM9DS1 over either SPI or I2C, so...
2.0.0 INSTALL

Output Serial Monitor X

Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.3, "kelembapan": 49.3}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.3, "kelembapan": 49.3}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.3, "kelembapan": 49.5}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.3, "kelembapan": 50}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.2, "kelembapan": 50.1}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.2, "kelembapan": 49.8}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.2, "kelembapan": 50.4}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.3, "kelembapan": 60.7}
[RIRIM MESH] {"node": "Node-2", "chipId": 3816590060, "suhu": 30.3, "kelembapan": 57.7}

Penjelasan Kode Khusus `painlessMesh` & Integrasi JSON:

- `Dht.readTemperature()` & `dht.readHumidity()` : Membaca nilai fisik suhu () dan kelembapan () dari sensor DHT11/DHT22.
- `serializeJson(doc, msg)` : Mengubah struktur objek JSON menjadi deretan teks string ringkas siap siar.
- `deserializeJson(doc, msg)` : Membedah string teks JSON yang masuk kembali menjadi variabel numerik terpisah.
- `painlessMesh mesh;` : Menginisialisasi objek kontrol jaringan mesh yang menangani pembentukan soft-AP dan koneksi STA secara simultan di latar belakang.
- `Mesh.getNodeId()` : Menghasilkan ID unik numerik berbasis MAC address bawaan chip ESP8266.
- `Mesh.sendBroadcast(msg)` : Menyiarkan paket data teks ke seluruh node di dalam jaringan mesh. Paket akan diteruskan (relayed) otomatis oleh node perantara jika node target berada di luar jangkauan radio langsung.
- `TaskScheduler (userScheduler)` : Pengganti fungsi `delay()`. Di dalam jaringan mesh, penggunaan `delay()` dilarang keras karena akan membekukan tugas latar belakang perutean paket (routing packet task), yang dapat menyebabkan runtuhnya topologi mesh.
- `Mesh.update()` : Menjalankan pemeliharaan koneksi, sinkronisasi jam internal mesh, dan pertukaran tabel rute secara kontinu di dalam siklus `loop()`.

5.6 Latihan

Latihan 1 (Eksperimen Deteksi ACK & Pemutusan Sempul – Praktikum 2): Saat program Praktikum 2 (ESP-NOW One-to-Many) sedang berjalan normal dan mencetak status “Berhasil Diterima” (`sendStatus == 0`), lakukan eksperimen pemutusan sempul:

Hasil Observasi Log Serial Monitor Pengirim (Controller):

Saat kabel USB NodeMCU 2 (Receiver) dicabut, log pada Serial Monitor Controller untuk alamat MAC NodeMCU 2 langsung berubah dari “Berhasil Diterima” (`sendStatus == 0`) menjadi “Gagal (Tidak Terjangkau)” (`sendStatus != 0`). Sementara itu, pengiriman ke NodeMCU 3 tetap bernilai “Berhasil Diterima”.

Analisis Protokol ESP-NOW pada Layer 2 MAC:

Meskipun tidak menggunakan transmisi TCP (tanpa connection-oriented handshake atau IP Layer), ESP-NOW bekerja pada Layer 2 (Data Link Layer) standar IEEE 802.11 menggunakan frame aksi (Vendor-Specific Action Frame). Pada tingkat hardware Wi-Fi, setiap kali frame data ESP-NOW dipancarkan ke alamat MAC tujuan, perangkat penerima secara otomatis mengirimkan balasan sinyal ACK (Acknowledgment) tingkat perangkat keras (Layer 2 MAC ACK) dalam hitungan fraksi milidetik.

Jika penerima dimatikan, tidak ada sinyal ACK yang kembali ke chip pengirim. Chip ESP8266 pengirim akan melakukan retry otomatis beberapa kali di tingkat physical/MAC layer. Jika tetap tidak ada respons ACK, hardware Wi-Fi memicu interupsi ke software yang mengembalikan status code gagal pada fungsi callback OnDataSent (sendStatus != 0).

Latihan 2: Eksperimen Dinamika Topologi & Self-Healing (painlessMesh)

Waktu Deteksi Pemutusan Simpul (changedConnectionCallback):

Hasil Observasi: Waktu yang dibutuhkan sekitar 3 hingga 10 detik. Penjelasan: painlessMesh menjaga topologi jaringan dengan mengirimkan paket keep-alive / heartbeat secara berkala di latar belakang. Ketika salah satu node dimatikan/di-RST, node tetangga akan kehilangan beberapa sinyal heartbeat berturut-turut. Setelah batas timeout terlampaui, node lain menyimpulkan bahwa simpul tersebut terputus dan memicu fungsi changedConnectionCallback() untuk memperbarui tabel routing.

Waktu Rekoneksi Otonom (Self-Healing):

Hasil Observasi: Waktu yang dibutuhkan dari saat daya disambungkan kembali hingga terdaftar otonom sekitar 5 hingga 12 detik. Penjelasan: Saat node booting ulang, antarmuka radio Wi-Fi-nya akan memindai channel dan mencari SSID/Mesh Prefix (LabIoTMesh) serta Password yang cocok. Begitu transmisi beacon saling merespons, mekanisme Self-Healing painlessMesh langsung mendaftarkan ulang node tersebut ke dalam topologi multi-hop tanpa perlu menyetel ulang konfigurasi jaringan secara manual.

Latihan 3: Modifikasi Filter Telemetry & Ambang Batas JSON (painlessMesh)

```
Blink | Arduino IDE 2.3.10
File Edit Sketch Tools Help

NodeMCU 1.0 (ESP-12...)

Blink.ino
1 #include <painlessMesh.h>
2 #include <DHT.h>
3 #include <ArduinoJson.h>
4
5 #define MESH_PREFIX "Lab5RSAR" // Nama jaringan Mesh bersama
6 #define MESH_PASSWORD "123" // Kata sandi jaringan Mesh
7 #define MESH_PORT 8080 // Port komunikasi TCP Mesh
8
9 // Konfigurasi pin dan tipe sensor DHT
10 #define DHTPIN 2 // Pin D4 (GPIO 2)
11 #define DHTTYPE DHT22 // Tipe sensor DHT22
12 DHT dht(DHTPIN, DHTTYPE);
13
14 // Identitas unik node untuk mempermudah identifikasi teks
15 const char* nodeName = "Node-1";
16
17 Scheduler userScheduler;
18 painlessMesh mesh;
19
20 // Prototipe fungsi pengiriman pesan berkala
21 void sendMessage();

Output Serial Monitor X
Message (Enter to send message to 'NodeMCU 1.0 (ESP-12E Module)' on 'COM3')

[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":30.8,"kelembapan":51.3}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":30.9,"kelembapan":51.2}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":30.9,"kelembapan":51.1}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":30.9,"kelembapan":50.9}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":30.9,"kelembapan":50.4}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":31,"kelembapan":50.1}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":30.9,"kelembapan":49.8}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":30.9,"kelembapan":49.8}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":31,"kelembapan":49.6}
[KIRIM MESH] {"node":"Node-1","chipId":3816590060,"suhu":31,"kelembapan":49.6}
```

1. Penjelasan Logika Program & Ambang Batas Program bawaan Praktikum 4 mencetak seluruh paket JSON yang masuk dari semua node pengirim tanpa penyaringan. Untuk mengubah salah satu board menjadi Pengawas Kondisi Suhu Kritis, dilakukan modifikasi pada fungsi callback receivedCallback: Menggunakan ArduinoJson untuk membedah (parsing) nilai suhu, node, chipId, dan kelembapan dari pesan string JSON. Menambahkan struktur percabangan if-else untuk mengevaluasi parameter suhu: Kondisi Kritis ($\text{suhu} > 32.0^\circ\text{C}$): Program mencetak kartu telemetry lengkap beserta peringatan status. Pengujian dapat dilakukan dengan mendekatkan jari hangat atau meniup sensor DHT22/DHT11 pada node pengirim hingga suhu naik melampaui 32.0°C . Kondisi Normal ($\text{suhu} \leq 32.0^\circ\text{C}$): Program hanya mencetak status singkat satu baris: [NORMAL] Telemetry dari Node <chipId> aman